WEB ENGINEERING

Assignment 1: Requirements

FEBRUARY 21, 2026

ARHAM JAN & ABDUL QAYYUM
01-131232-013 & 01-131232-003

DR-01: ACID-Compliant Financial Wallet States

- **Description:** The system shall maintain a self-contained double-entry ledger to track user financial states. It must continuously monitor three distinct values for every user: the funds currently available for bidding, the funds actively locked in escrow, and the total net deposits.
- **Rationale:** To prevent "double-spend" vulnerabilities during the simulation phase, ensuring financial accuracy without relying on an external third-party payment gateway.
- **Acceptance Criteria:**
    - *Test 1:* The mathematical sum of a user's available funds and locked funds must always exactly equal their total deposited funds. Any transaction attempt that violates this equation must be hard-rejected.

DR-02: Concurrent Bid Atomic Commits

- **Description:** The system must guarantee atomic data operations during bid evaluations. If multiple bid payloads for the same auction arrive simultaneously, the data layer must process them sequentially to ensure no data is overwritten.
- **Rationale:** In high-frequency, last-second auction scenarios, race conditions can cause a lower bid to overwrite a higher bid. Sequential locking guarantees only the mathematically highest valid bid is written to the active ledger.
- **Acceptance Criteria:**
    - *Test 1:* Injecting 500 simultaneous bid payloads for the same auction identifier must result in exactly 1 winning bid recorded, with the other 499 correctly logged as rejected or outbid.

DR-03: Unidirectional Escrow Lifecycle Enforcement

- **Description:** The data layer shall enforce a strict, unidirectional state machine for all active listings. A transaction must only transition through predefined legal states (e.g., `Available` → `Awaiting Meetup` → `Funds Locked` → `Settled` or `Disputed`).
- **Rationale:** To prevent logic bypass vulnerabilities where a malicious user or system bug attempts to force a transaction into a settled state without passing through the required validation steps.
- **Acceptance Criteria:**
    - *Test 1:* The system must throw a fatal state transition error if an operation attempts to move an item from the 'Available' state directly to the 'Settled' state.

DR-04: KYC & Regulatory Identity Data

- **Description:** The system must securely store seller identity profiles, explicitly mandating the storage of a valid National Tax Number (NTN) and a dynamically calculated user trust score.
- **Rationale:** To comply with the FBR E-Commerce regulations which bar unregistered sellers from utilizing online marketplaces.
- **Acceptance Criteria:**
    - *Test 1:* The system must reject the creation of a new seller profile if the NTN field is null or fails standard format validation.

DR-05: Immutable Taxation Audit Trails

- **Description:** Upon the successful settlement of an escrow transaction, the data layer must automatically generate distinct, immutable ledger entries for all applicable tax deductions (e.g., 1% Withholding Tax, 15% ICT Sales Tax) and platform fees.
- **Rationale:** To ensure legal compliance and provide exact financial reporting capabilities when auditing the platform's revenue versus government tax liabilities.
- **Acceptance Criteria:**
    - *Test 1:* A successfully settled transaction must automatically log the exact calculated tax deductions strictly based on the platform's isolated fee structure.

DR-06: Cryptographic QR Seed Storage

- **Description:** The cryptographic seeds used to generate the time-sensitive QR codes for the physical meetup must be encrypted at rest within the data store using industry-standard symmetric encryption.
- **Rationale:** If the backend data store is compromised, plaintext seeds would allow an attacker to mathematically generate valid QR codes and steal escrowed funds without ever meeting the buyer.
- **Acceptance Criteria:**
    - *Test 1:* A direct read of the underlying storage mechanism for meetup secrets must return unreadable ciphertext, not the raw seed string.

DR-07: Append-Only Transaction Immutability

- **Description:** The system shall enforce an append-only architecture for all financial history and bidding ledgers. Existing records cannot be modified or removed. Any state change or reversal must be logged as a new, counter-acting append transaction.
- **Rationale:** To maintain a flawless audit trail that can resolve disputes if a buyer or seller claims a transaction was manipulated.
- **Acceptance Criteria:**
    - *Test 1:* Any system-level or API-level command attempting a modification or deletion operation on the financial history ledger must return a hard failure.

DR-08: AI Metadata Sanitization

- **Description:** The system shall extract metadata (specifically GPS coordinates and device timestamps) from uploaded images and store it in an isolated, restricted data structure for AI verification. The system must permanently strip this metadata from the primary image file before storing it for public retrieval.
- **Rationale:** To protect the physical safety and privacy of the sellers by preventing location tracking from public marketplace listings.
- **Acceptance Criteria:**
    - *Test 1:* Retrieving an image directly from the public auction gallery and inspecting it with a metadata viewer must yield zero geolocation data.

DR-09: High-Velocity Bid Caching

- **Description:** The data architecture must implement a high-velocity caching layer to ingest and log microsecond-level bid histories, which are then asynchronously aggregated and flushed to the persistent, long-term data store.
- **Rationale:** Writing thousands of rapid-fire bids directly to a persistent database in the last 10 seconds of an auction causes catastrophic locks. Caching ensures the system survives traffic spikes.
- **Acceptance Criteria:**
    - *Test 1:* The persistent data store must accurately reflect the final highest bid even if thousands of bids are ingested by the caching layer within a single 2-second window.

DR-10: Dispute Data Archiving

- **Description:** If an auction ends in a disputed state, the data layer must instantly freeze all associated records (chat logs, biometric hashes, QR scan attempts) and archive them in a restricted domain accessible only to authorized administrators.
- **Rationale:** To preserve forensic evidence of fraud, theft, or physical duress for potential investigation without allowing users to alter the evidence.
- **Acceptance Criteria:**
    - *Test 1:* Once a transaction is flagged as disputed, any user-level operation attempting to alter, delete, or hide associated data must be blocked.

DR-11: Two-Phase Commit (2PC) Settlement

- **Description:** The data layer shall execute the final settlement using a strict Two-Phase Commit protocol. In the "Prepare Phase," the system must pre-calculate all taxes, verify all balances, and place a hard lock on the relevant data rows. Only when the Prepare Phase returns a 100% success state will the system enter the "Commit Phase" to permanently write the settlement and broadcast the "Success" signal to the frontend client.
- **Rationale:** To completely eliminate the need for dangerous post-scan rollbacks. By verifying the exact mathematical and system state *before* finalizing the transaction, the platform guarantees that if a system failure occurs, it happens *before* the seller hands over the physical item, not after.
- **Acceptance Criteria:**
    - *Test 1:* A simulated database or network failure during the "Prepare Phase" must gracefully abort the transaction and return a "Scan Failed - Do Not Hand Over Item" error to the users, leaving the escrow lock entirely intact.

DR-12: Cryptographic Non-Repudiation (The Legal Shield)

- **Description:** Upon a successful QR scan, the data layer must generate and store a unique, immutable cryptographic receipt (a hash combining the buyer's ID, seller's ID, item ID, exact timestamp, and geolocation).
- **Rationale:** To provide absolute legal protection against chargebacks or fraud. Non-repudiation means neither the buyer nor the seller can legally deny that the transaction took place. If a buyer calls their bank to claim "I never received the item," Boli.pk has the mathematical proof that their specific device scanned the seller's specific code at that exact physical location.
- **Acceptance Criteria:**
    - *Test 1:* The generated settlement hash must be mathematically verifiable using the platform's public key, and any alteration to the underlying transaction data must immediately invalidate the hash.

DR-13: Multi-Signature Mutual Cancellation

- **Description:** Escrowed funds must never be unilaterally reversed or "timed out" by the system based purely on a timer. If an active meetup window expires, the data layer must transition the transaction into a frozen Arbitration state. Reversing the funds back to the buyer requires either a dual-signature (both buyer and seller confirm the deal is off via the app) or an administrative override.
- **Rationale:** To prevent temporal fraud. If the system automatically refunded the buyer after 72 hours (as previously drafted), a malicious buyer could delay the seller, take the item at hour 71, and wait for the system to automatically refund them an hour later. Mutual cancellation ensures absolute consensus.
- **Acceptance Criteria:**
    - *Test 1:* If a transaction reaches its expiration timestamp without a QR scan, the database must reject any automated withdrawal attempt and require explicit cryptographic consent from the seller's account to release the funds back to the buyer.

DR-14: Zero-Bid Expiry Lifecycle

- **Description (What):** The data layer must autonomously detect when an auction timer reaches zero with exactly zero recorded bids and instantly transition the listing to an UNSOLD_EXPIRED state, completely bypassing all escrow routing logic.
- **Rationale (Why):** The system must not attempt to route an empty auction into the settlement phase, which would trigger null-pointer exceptions in the Two-Phase Commit logic and corrupt the daily FBR tax calculations.
- **Acceptance Criteria:** An auction reaching its exact expiration timestamp with an empty bid array must immediately release the database row lock and render a "Relist Item" prompt exclusively on the seller's dashboard.

Communications Interfaces (Network)

IR-01: Full-Duplex Connection Establishment

- **Description (What):** The client application must establish a persistent, full-duplex connection to the backend gateway to receive live auction state updates.
- **Rationale (Why):** To enable server-pushed updates, eliminating the network overhead and latency of traditional client-side polling.
- **Acceptance Criteria:**
  - *Test 1:* Upon entering a bidding room, the client must successfully open a WSS (WebSocket Secure) connection, and the server must register the active session within 100 milliseconds.

IR-02: Connection Heartbeat Protocol

- **Description (What):** The communications protocol must enforce a bi-directional keep-alive heartbeat at predefined intervals (e.g., every 30 seconds).
- **Rationale (Why):** To actively detect dead or stalled connections where the network drops but the operating system fails to notify the application.
- **Acceptance Criteria:**
  - *Test 1:* If the server explicitly blocks the client's "pong" response, the backend must automatically sever the dead connection and free the memory after exactly 30 seconds.

IR-03: Autonomous Reconnection Strategy

- **Description (What):** Upon detecting a dropped connection, the client interface must autonomously execute an exponential backoff reconnection sequence (e.g., 1s, 2s, 4s) without requiring user input.
- **Rationale (Why):** To ensure users automatically recover from momentary mobile data dead zones (like driving under a bridge) during an active auction.
- **Acceptance Criteria:**
  - *Test 1:* Simulating a total network drop on the client device for 5 seconds must result in the app silently reconnecting and fetching the latest bid state the moment the network is restored, without the user tapping a "Reconnect" button.

IR-04: Out-of-Band Notification Delivery

- **Description (What):** The system must interface with native mobile push-notification services to deliver critical financial alerts (e.g., "Outbid", "Funds Released").
- **Rationale (Why):** Users must receive critical state changes even when the application is backgrounded or the device is locked.
- **Acceptance Criteria:**
  - *Test 1:* Triggering an "Outbid" database event for a user whose application is explicitly closed must result in a native OS push notification appearing on their lock screen.

IR-05: Notification Latency Threshold

- **Description (What):** The backend must dispatch out-of-band notifications to the provider within a strict latency threshold (e.g., under 2000 milliseconds) from the moment the triggering database commit occurs.
- **Rationale (Why):** A delayed "Outbid" notification in the final minute of an auction prevents the user from reacting, causing unfair financial loss.
- **Acceptance Criteria:**
  - *Test 1:* The measured time between the PostgreSQL commit for a new highest bid and the API payload dispatch to the APNs/FCM notification server must not exceed 2000 milliseconds.

Software Interfaces (APIs)

IR-06: Asynchronous Vetting API Boundary

- **Description (What):** The core routing backend must interface with the isolated computer-vision microservice via an internal, firewall-protected API for image analysis.
- **Rationale (Why):** To isolate CPU-heavy image processing from the high-speed bidding architecture, preventing traffic spikes from crashing the entire platform.
- **Acceptance Criteria:**
  - *Test 1:* Transmitting a valid multipart image payload from the core server to the internal AI endpoint must successfully return a JSON object containing a numeric "Risk Score".

IR-07: AI Service Timeout Constraint

- **Description (What):** The internal API gateway must enforce a strict maximum timeout threshold (e.g., 5000 milliseconds) on all requests sent to the AI microservice.
- **Rationale (Why):** To prevent pending API requests from hanging indefinitely and consuming all available backend connection threads if the AI service stalls.
- **Acceptance Criteria:**
    - *Test 1:* Artificially delaying the AI microservice response by 10 seconds must cause the core backend gateway to explicitly terminate the connection at exactly 5000 milliseconds.

IR-08: AI Service Graceful Degradation

- **Description (What):** If the AI microservice interface times out or returns a 5xx server error, the gateway must gracefully default the listing to a "Manual Review Required" state rather than dropping the user's upload.
- **Rationale (Why):** To maintain platform uptime and user experience even during partial microservice outages.
- **Acceptance Criteria:**
    - *Test 1:* Simulating an HTTP 503 Service Unavailable response from the AI microservice must result in the successful creation of the database listing, flagged strictly for "Admin Manual Review".

IR-09: Cryptographic Webhook Validation

- **Description (What):** Any external API endpoint exposed to receive webhook payloads (e.g., payment success signals) must strictly validate the payload using a Hash-Based Message Authentication Code (HMAC).
- **Rationale (Why):** To prevent malicious actors from spoofing payment confirmations and tricking the system into releasing escrowed funds.
- **Acceptance Criteria:**
    - *Test 1:* Transmitting a properly formatted "Payment Success" webhook payload with an intentionally invalid HMAC signature must immediately return an HTTP 401 Unauthorized status and log a critical security event.

Hardware Interfaces (Device Sensors)

IR-10: Primary Optical Hardware Access

- **Description (What):** The client application must interface with the device's native media APIs to explicitly request and activate the environment-facing (rear) camera lens.
- **Rationale (Why):** To ensure the QR scanning interface automatically opens the correct hardware sensor for scanning another user's screen.
- **Acceptance Criteria:**
    - *Test 1:* Navigating to the "Scan QR" route for the first time must trigger the native iOS/Android operating system prompt requesting camera permissions.

IR-11: Optical Stream Processing Rate

- **Description (What):** The hardware interface must process the incoming optical video stream at a minimum rate of 15 frames per second to decode the cryptographic QR payload.
- **Rationale (Why):** To guarantee high-speed, frictionless scanning even on older devices or in suboptimal lighting conditions during a physical meetup.
- **Acceptance Criteria:**
    - *Test 1:* Supplying a standard-resolution video feed to the scanner interface must yield a measured processing rate of greater than or equal to15 FPS on the client's rendering thread.

IR-12: Hardware Permission Fallback

- **Description (What):** If the operating system or user denies access to the camera hardware, the application must immediately render a manual entry fallback interface for a cryptographic PIN.
- **Rationale (Why):** A hardware permission denial must not permanently lock a user out of settling a transaction and retrieving their funds.
- **Acceptance Criteria:**
    - *Test 1:* Explicitly denying camera permissions at the OS level must instantly replace the black camera viewfinder UI with a numeric PIN entry form.

IR-13: Native Biometric Enclave Trigger

- **Description (What):** Prior to rendering the settlement QR code, the application must interface with the device's native biometric hardware enclave to request a fingerprint or facial scan.
- **Rationale (Why):** To cryptographically ensure the person holding the physical device is the authorized owner before generating the financial handshake.
- **Acceptance Criteria:**
  - *Test 1:* Tapping "Generate Settlement Code" must pause the application state and trigger the native FaceID/TouchID prompt. The QR code must remain hidden until a "Match Success" boolean is returned by the OS.

IR-14: Dynamic Hardware Brightness Control

- **Description (What):** Upon rendering the QR cryptogram on the screen, the application must interface with the device's display hardware to temporarily maximize screen brightness.
- **Rationale (Why):** To ensure the QR code is scannable by the opposing user's camera even in direct, bright outdoor sunlight.
- **Acceptance Criteria:**
  - *Test 1:* Entering the QR display route must override the user's manual brightness setting, forcing it to 100%, and exiting the route must restore the exact previous brightness level.

User Interfaces (Frontend)

IR-15: Optimistic UI State Transition

- **Description (What):** Upon a user submitting a bid interaction, the UI must immediately render a visual "Pending" state without waiting for the server response.
- **Rationale (Why):** To mask unavoidable network latency and prevent the user from assuming the button press failed and spam-clicking the interface.
- **Acceptance Criteria:**
  - *Test 1:* Clicking the "Place Bid" button must trigger a visual loading state within 16 milliseconds (1 frame), well before the WebSocket round-trip completes.

IR-16: Tactile Haptic Feedback Integration

- **Description (What):** The UI must trigger the device's native vibration motor (haptic feedback) the exact moment a high-stakes interaction (like submitting a bid or scanning a QR code) is initiated.
- **Rationale (Why):** To provide physical confirmation of an action, reducing user anxiety during tense financial interactions.
- **Acceptance Criteria:**
    - *Test 1:* Executing a bid submission must call the native Navigator.vibrate() API for a duration of exactly 50 milliseconds.

IR-17: UI Conflict Resolution Reversal

- **Description (What):** If the optimistic "Pending" state receives a rejection payload from the server (e.g., the user was outbid by milliseconds), the UI must instantly revert to a highly visible "Outbid" error state.
- **Rationale (Why):** The interface must never leave a user under the false impression that they hold the winning bid if the backend data layer rejected it.
- **Acceptance Criteria:**
    - *Test 1:* Injecting an HTTP 409 Conflict response while the UI is in the "Pending" state must immediately abort the pending animation and render a red "Outbid" status block.

IR-18: Accessibility Contrast Compliance

- **Description (What):** The graphical interface designated for the physical meetup phase must adhere to strict WCAG 2.1 AAA contrast ratios (minimum 7:1 for text against backgrounds).
- **Rationale (Why):** To guarantee legibility and usability for all users, regardless of visual impairments or challenging environmental lighting.
- **Acceptance Criteria:**
    - *Test 1:* Running a standardized WCAG contrast analyzer against the Meetup UI screen must return a pass result for all textual elements at the AAA (7:1) threshold.

IR-19: Silent Duress Authentication (The Wrench Attack)

- **Description (What):** The system must support a secondary "Duress PIN" that, when entered during the physical meetup phase, authenticates the user but silently routes the transaction into a locked DISPUTED_DURESS state while rendering a superficial "Success" UI on the device.
- **Rationale (Why):** To protect users from physical harm if coerced into unlocking their device to release high-value escrow funds in a parking lot.
- **Acceptance Criteria:** Entering the pre-configured Duress PIN must yield a green "Payment Processing" screen on the frontend while simultaneously firing a high-priority webhook to platform admins and permanently freezing the backend ledger.

IR-20: Asymmetric Offline Challenge (The Dead Battery Fallback)

- **Description (What):** The settlement engine must provide a fallback cryptographic SMS challenge generator for the seller, allowing the buyer to authorize the fund release via a standard text message from an unauthenticated device if their primary smartphone loses power.
- **Rationale (Why):** Physical meetups are vulnerable to hardware failures. The transaction must not become permanently deadlocked, and the seller must not be left stranded if the buyer's phone dies and cannot display the QR code.
- **Acceptance Criteria:** The seller selecting "Buyer Device Offline" must generate a 6-digit cryptographic challenge. The system must release the escrowed funds strictly upon receiving this exact code via an inbound SMS from the buyer's FBR-registered phone number.

Authentication & Authorization Routing

NR-01: Unauthenticated Route Interception

- **Description (What):** The client application must intercept any attempt to access a protected transactional route without a valid cryptographic session token and immediately redirect the user to the authentication interface.
- **Rationale (Why):** To prevent anonymous actors from viewing sensitive escrow states or attempting to inject payloads into active bidding rooms.
- **Acceptance Criteria:**
    - *Test 1:* Directly entering a secure URL (e.g., a specific escrow room) into an unauthenticated browser must result in a forced redirect to the login route within 50 milliseconds.

NR-02: Post-Authentication Intent Resolution

- **Description (What):** When the system intercepts an unauthenticated request and redirects the user to the login route, it must temporarily store the original intended URI and automatically route the user back to that exact URI immediately upon successful authentication.
- **Rationale (Why):** If a user clicks an "Outbid" email link but isn't logged in, forcing them to navigate back to the home page after logging in destroys the user experience and guarantees they miss the bid.
- **Acceptance Criteria:**
    - *Test 1:* A user intercepted while trying to access "Auction A", who then successfully logs in, must land directly on "Auction A" without seeing the general dashboard.

NR-03: Regulatory Funneling (KYC Routing)

- **Description (What):** If an authenticated user attempts to navigate to the "Create Listing" route without a verified National Tax Number (NTN) attached to their profile, the system must intercept the navigation and route them strictly to the FBR Verification onboarding flow.
- **Rationale (Why):** To enforce legal compliance structurally. A user cannot be allowed to view the listing creation tools if they are legally barred from selling.
- **Acceptance Criteria:**
    - *Test 1:* Clicking the "Sell Item" button on an account with a null NTN field must push the "Tax Setup" route to the view stack.

NR-04: Cryptographic Session Expiration Routing

- **Description (What):** The client interface must actively monitor the Time-To-Live (TTL) of the user's session token. If the token expires while the user is actively viewing a secure route, the system must immediately purge the view and force a redirect to the "Session Expired" authentication screen.
- **Rationale (Why):** To prevent users from viewing stale, cached financial data or attempting to execute transactions on a dead session, which would result in cascading backend errors.
- **Acceptance Criteria:**
    - *Test 1:* Artificially expiring a session token while the user is actively viewing their Escrow Dashboard must force a redirect to the login screen within 1 second.

NR-05: Concurrent Session Override Routing

- **Description (What):** If the system detects a new authentication event for a user on Device B, any active, secure routes currently open on Device A must be instantly invalidated and routed to a "Session Terminated: Logged in elsewhere" screen.
- **Rationale (Why):** To prevent race conditions and fraud where a single user attempts to manipulate an auction or escrow state from two different devices simultaneously.
- **Acceptance Criteria:**
    - *Test 1:* Logging into the same account on a desktop browser must instantly force the active mobile application session to route to the termination screen.

State-Driven Transactional Routing

NR-06: Autonomous Expiry Redirection (The Winner)

- **Description (What):** At the exact millisecond an auction timer reaches zero, the interface must autonomously route the *winning* bidder from the public bidding room directly to the secure "Escrow Initialization" route.
- **Rationale (Why):** To create a frictionless transition into the Two-Phase Commit escrow lock, ensuring the winner secures their funds immediately without needing to hunt for a "Pay Now" button.
- **Acceptance Criteria:**
  - *Test 1:* When the server broadcasts the AUCTION_ENDED event, the client identified as the highest bidder must have their view automatically pushed to the settlement flow.

NR-07: Autonomous Fallback Redirection (The Losers)

- **Description (What):** At the exact millisecond an auction timer reaches zero, the interface must autonomously route all *losing* bidders from the bidding room to a "Similar Active Auctions" fallback route.
- **Rationale (Why):** Keeping losing bidders in a dead room is a dead-end user journey. Routing them to similar items immediately captures their active purchasing intent and keeps money moving through the platform.
- **Acceptance Criteria:**
  - *Test 1:* When the server broadcasts the AUCTION_ENDED event, any client not identified as the highest bidder must be routed to a curated gallery of similar active listings.

NR-08: Isolated Arbitration Confinement

- **Description (What):** If a transaction state changes to DISPUTED, the system must immediately revoke both the buyer's and seller's access to all standard settlement routes and autonomously redirect them to an isolated "Arbitration Chat" route.
- **Rationale (Why):** Once a dispute is initiated, users must be completely blocked from navigating back to the QR scanner or fund-release interfaces to prevent them from attempting to force a settlement outside of admin supervision.
- **Acceptance Criteria:**
  - *Test 1:* Attempting to manually load the QR generation route for a DISPUTED transaction ID must fail and redirect the user to the Arbitration view.

NR-09: Administrative Quarantine Routing

- **Description (What):** If the internal AI API assigns a "High Risk" score to a newly submitted listing, the routing controller must bypass the public marketplace entirely and route the listing entity directly into the internal Administrative Review queue.
- **Rationale (Why):** Fraudulent items must never touch the public navigation tree.
- **Acceptance Criteria:**
    - *Test 1:* A listing finalized with an AI risk score of 85 must return a 404 Not Found if accessed via the public gallery URI, but must render successfully via the Admin portal URI.

History, Deep Linking, & Edge Cases

NR-10: Post-Settlement History Invalidation

- **Description (What):** Upon the successful completion of the QR cryptographic handshake (the fund release), the application must invalidate the routing history stack, preventing the user from using the native device "Back" button to return to the active QR scanner or Escrow lock screens.
- **Rationale (Why):** Returning to a completed transactional screen causes user confusion and can trigger accidental duplicate API requests that the backend then has to explicitly reject.
- **Acceptance Criteria:**
    - *Test 1:* Pressing the Android hardware "Back" button or swiping back on iOS immediately after viewing the "Payment Successful" receipt must route the user to their main dashboard, not back to the camera scanner.

NR-11: Destructive Action Interruption

- **Description (What):** The navigation controller must intercept any attempt to navigate away (via back button, tab close, or URI change) from a critical financial flow (e.g., mid-way through the Escrow lock) and trigger a mandatory confirmation prompt.
- **Rationale (Why):** To prevent catastrophic data loss or transaction abandonment caused by an accidental swipe or misclick during high-stakes moments.
- **Acceptance Criteria:**
    - *Test 1:* Triggering a native OS "Back" gesture while the Two-Phase Commit is pending user confirmation must halt the navigation and render a "Cancel Transaction?" warning prompt.

NR-12: Universal Deep Link Payload Parsing

- **Description (What):** The application router must be capable of intercepting universal deep links (e.g., bolipk://auction/12345) from native OS push notifications, parsing the embedded entity IDs, and routing the user directly to the specific internal state.
- **Rationale (Why):** To ensure users tapping on an "Outbid" notification are taken instantly to the battleground, rather than a generic home screen.
- **Acceptance Criteria:**
    - *Test 1:* Tapping a specific push notification while the app is closed must launch the application and land the user exactly on the requested auction room within the initial render cycle.

NR-13: Invalid Resource Fallback (The 404 Guard)

- **Description (What):** If a user attempts to navigate to an entity ID (like an Auction ID or User Profile) that does not exist or has been deleted, the system must catch the invalid parameter and route the user to a contextual "Resource Unavailable" screen rather than crashing the interface.
- **Rationale (Why):** Standard error handling. The frontend application must never break or display raw code errors if a user alters a URL parameter manually.
- **Acceptance Criteria:**
    - *Test 1:* Navigating to /auction/invalid_id_999 must cleanly render a formatted 404 layout with a button returning the user to the marketplace.

NR-14: Stale State Re-Routing

- **Description (What):** If a user clicks a link to an auction that was active when the page loaded but has since been completed or cancelled by the seller, the router must intercept the request, fetch the updated state, and route the user to the "Auction Ended" receipt view instead of the live bidding room.
- **Rationale (Why):** To prevent users from entering a bidding room where the WebSocket connection will immediately be rejected by the server due to a closed state.
- **Acceptance Criteria:**
    - *Test 1:* Clicking a bookmark for an auction that ended 5 minutes ago must route the user directly to the historical price receipt, not the active bidding layout.

NR-15: Offline Route Resolution

- **Description (What):** If the client device loses all network connectivity, the routing engine must intercept any new navigation attempts and serve a cached, dedicated "Offline Mode" route rather than allowing the browser or OS to display a generic "No Internet" dinosaur screen.
- **Rationale (Why):** Maintaining brand immersion and user control even when the hardware fails. The offline route can provide instructions or cached historical data.
- **Acceptance Criteria:**
    - *Test 1:* Disabling Wi-Fi and Cellular data and attempting to click a new category link must instantly render the platform's custom "Network Disconnected" UI layout.

Security & Trust Personalization

PR-01: Trust-Weighted Security Friction

- **Description (What):** The system shall enforce dynamic authentication thresholds during high-value bid submissions. Users with a calculated trust score below a predefined threshold must be challenged with Multi-Factor Authentication (MFA) before the bid is ingested, whereas top-tier verified users must bypass this friction for identical actions.
- **Rationale (Why):** To balance platform security with user conversion. Unproven users require strict verification to prevent hit-and-run fraud, while proven, high-volume buyers must be allowed to execute split-second bids without being bottlenecked by an OTP SMS.
- **Acceptance Criteria:**
    - *Test 1:* A user with a Trust Score of 20 attempting a Rs. 100,000 bid must successfully resolve an MFA prompt before the payload is sent. A user with a Trust Score of 95 executing the exact same bid must have the payload transmitted instantly with zero interruptions.

PR-02: Asymmetric Reputation Visibility

- **Description (What):** The user profile component must dynamically reconstruct its visible data fields depending on the active transactional role of the viewer.
- **Rationale (Why):** Different roles require different trust metrics to feel confident. A buyer needs to see a seller's AI vetting pass rate and authenticity guarantee. A seller needs to see a buyer's escrow funding reliability and no-show rate at physical meetups.
- **Acceptance Criteria:**
    - *Test 1:* Viewing User A's profile from a "Seller" context must render their escrow completion percentage, whereas viewing User A from a "Buyer" context must hide that metric and instead render their item authenticity rating.

State-Driven UI Personalization

PR-03: Context-Aware Dashboard Instantiation

- **Description (What):** The application must dynamically reconstruct the user's primary landing dashboard based on their most critical real-time financial state, strictly prioritizing active "Awaiting Meetup" or "Action Required" entities above standard market feeds.
- **Rationale (Why):** If a user is walking into a parking lot to hand over an item, they should not have to dig through menus to find the QR scanner. The system must anticipate the user's physical context and push the most urgent financial tool to the top of the UI stack.
- **Acceptance Criteria:**
    - *Test 1:* Authenticating into the application while possessing an active transaction in the AWAITING_MEETUP state must result in the Escrow Scanner interface rendering as the absolute top-level, full-width dashboard component.

PR-04: Dynamic Bid-Increment Suggestions

- **Description (What):** The bidding interface must dynamically calculate and render personalized quick-bid values. These values must be mathematically derived from the current highest bid and the auction's historical velocity, and strictly capped by the user's currently available wallet balance.
- **Rationale (Why):** To remove cognitive friction during the final 10 seconds of a high-stress auction. Users must be able to safely escalate their bids with a single tap without the system suggesting an increment that would result in an "Insufficient Funds" rejection.
- **Acceptance Criteria:**
    - *Test 1:* If the current bid is Rs. 50,000 and the user only has Rs. 55,000 in available funds, the system must not render a "Quick Bid + Rs. 10,000" button, strictly limiting the maximum suggested one-tap bid to Rs. 55,000.

Feed & Notification Personalization

PR-05: Historical Affinity Market Feed

- **Description (What):** The public marketplace gallery must algorithmically sort and prioritize listings based on the user's explicit interaction history (e.g., categories of previously placed bids, watched items, and AI-vetted asset classes).
- **Rationale (Why):** To maximize bid volume and liquidity. The system must immediately present high-intent buyers with the exact asset classes they have mathematically proven a willingness to purchase.
- **Acceptance Criteria:**
    - *Test 1:* A user whose last three bids were on gaming laptops must see newly vetted laptop listings injected into the first three positions of their default marketplace feed upon their next session load.

PR-06: Granular Notification Threshold Configuration

- **Description (What):** The interface must provide granular configuration tools allowing users to define atomic, personalized thresholds for out-of-band alerts (e.g., silencing outbid alerts during sleep hours, or only sending a push notification if outbid by a margin greater than 5%).
- **Rationale (Why):** Alert fatigue kills engagement. If a user receives 50 push notifications for Rs. 100 micro-bids on a Rs. 200,000 item, they will disable notifications entirely—causing them to miss the critical alert when the auction actually ends.
- **Acceptance Criteria:**
    - *Test 1:* A user configuring a "> 5% Outbid Alert" on a Rs. 10,000 bid must absolutely not receive a push notification if an opposing user places a Rs. 10,200 bid.

PR-07: Localized Accessibility Override Retention

- **Description (What):** The application must persistently store the user's specific accessibility overrides (e.g., forced high-contrast mode, disabled tactile haptics) in the backend database and apply them instantly across any authenticated device.
- **Rationale (Why):** True personalization includes physical usability. Forcing a visually impaired user to reconfigure their reading settings every time their session token expires creates an unacceptable barrier to entry for financial transactions.
- **Acceptance Criteria:**
    - *Test 1:* Enabling high contrast mode on a mobile device, intentionally terminating the session, and logging in via a distinct desktop browser must immediately render the desktop interface in the requested high contrast state without further user input.

Environmental & Network Personalization

PR-08: Network-Adaptive Payload Delivery

- **Description (What):** The client application must dynamically query the device's current network connection API. If the user is on a throttled or unstable connection, the system must autonomously personalize the data payload by serving heavily compressed media assets and disabling heavy UI animations.
- **Rationale (Why):** Mobile data can be highly inconsistent. If a buyer enters a bidding room on a weak 3G connection and the system tries to load uncompressed 4K images, their device will lag, and they will lose the auction.
- **Acceptance Criteria:**
    - *Test 1:* Throttling the client network to a "Slow 3G" profile must trigger the backend to exclusively serve thumbnail-resolution image URIs instead of the full-resolution assets for the marketplace feed.

PR-09: Temporal Risk Warnings (Time-of-Day Context)

- **Description (What):** The system must personalize the physical meetup scheduling interface based on the user's local timezone. If a user attempts to initiate or schedule an escrow QR handshake during late-night hours, the interface must inject a mandatory, high-visibility warning prompt.
- **Rationale (Why):** Systemic safety. Meeting strangers in a parking lot at 2:00 AM for high-value asset exchange is a massive physical security risk. The system must adapt its UI to warn users against high-risk environmental contexts.
- **Acceptance Criteria:**
    - *Test 1:* Attempting to transition a listing to the AWAITING_MEETUP state at 11:30 PM local time must freeze the UI transition and render a red confirmation modal requiring the user to explicitly type "I AGREE" before proceeding.

Financial Tier Personalization

PR-10: KYC-Tiered Velocity Limits

- **Description (What):** The system must dynamically adjust a user's rolling 24-hour financial transaction limit based on the depth of their KYC (Know Your Customer) profile. A user with basic verification must be hard-capped at a lower bidding velocity compared to a user with full biometric and tax verification.
- **Rationale (Why):** To strictly limit the platform's exposure to organized fraud rings. A new, unverified account should not be physically capable of bidding on massive volumes of high-end assets in a single afternoon.
- **Acceptance Criteria:**
    - *Test 1:* A "Tier 1" (Basic Verification) user attempting to place a bid that pushes their 24-hour escrow exposure above their specific tier limit must have the bid rejected with a prompt to upgrade to "Tier 2" (Full Verification).

PR-11: Adaptive Language & Locale Injection

- **Description (What):** The application must intercept the native OS locale preference upon the first boot. If the OS locale matches a supported regional language (e.g., Urdu), the system must autonomously serve the localized dictionary for all UI text, transactional receipts, and push notifications without requiring manual configuration.
- **Rationale (Why):** Financial platforms require absolute comprehension. Forcing a user to navigate complex legal or financial disclaimers in a non-native language during an escrow lock is unethical and leads to costly user errors.
- **Acceptance Criteria:**
    - *Test 1:* Launching the application on a device with a local language OS setting must immediately render the primary authentication and dashboard buttons in localized text, overriding the default fallback language.

Phase 1: Mathematical & Ledger Absolute Integrity

TR-01: Integer-Based Currency Storage (Zero Floating-Point)

- **Description (What):** The system shall strictly store, transmit, and compute all financial values exclusively as integers representing the lowest denomination of the currency (e.g., Paisas, not Rupees).
- **Rationale (Why):** Floating-point arithmetic (e.g., decimals like 100.50) causes microscopic precision errors in computer memory. Over millions of transactions, these decimal fractions "leak," creating massive accounting discrepancies.
- **Acceptance Criteria:**
    - *Test 1:* The database schema and all API payloads must reject any financial integer containing a decimal point, strictly enforcing a 10050 (Paisa) payload for a Rs. 100.50 transaction.

TR-02: Zero-Sum Double-Entry Constraint

- **Description (What):** The transactional engine must enforce a strict zero-sum equation across the entire platform's global ledger. Every single transaction must contain equal and offsetting debit and credit entries.
- **Rationale (Why):** Money cannot be created or destroyed by the platform. If a user's wallet goes down by Rs. 1,000, exactly Rs. 1,000 must appear across the Escrow, Tax, and Fee ledgers.
- **Acceptance Criteria:**
    - *Test 1:* Before committing any database transaction, a pre-commit trigger must sum the total debits and credits of the operation. If the result is not exactly 0, the entire transaction block must fatally abort.

TR-03: Fractional Dust Allocation

- **Description (What):** In the event that a percentage-based tax or fee calculation results in a fractional remainder (e.g., a 15% tax generating an uneven Paisa split), the system must utilize Banker's Rounding and automatically route the fractional variance (the "dust") to a dedicated, audited platform reconciliation account.
- **Rationale (Why):** If the rounding variance is not explicitly accounted for, the Zero-Sum ledger (TR-02) will fail to balance at the end of the day.
- **Acceptance Criteria:**
    - *Test 1:* A fee calculation that mathematically results in Rs. 10.005 must credit Rs. 10.00 to the fee account and exactly 0.5 Paisa equivalents to the platform dust ledger.

Phase 2: Concurrency & Attack Prevention

TR-04: Pessimistic Row Locking (TOCTOU Prevention)

- **Description (What):** The data layer must enforce strictly ordered, pessimistic locking (SELECT ... FOR UPDATE) on the user's wallet balance during any financial mutation to prevent Time-of-Check to Time-of-Use (TOCTOU) race conditions.
- **Rationale (Why):** A hacker with Rs. 10,000 could write a script to simultaneously submit 50 bids of Rs. 10,000 in the same millisecond. If the system checks the balance for all 50 before deducting the first one, the user spends Rs. 500,000 they don't have.
- **Acceptance Criteria:**
    - *Test 1:* Firing 100 concurrent withdrawal requests for a user's total balance must result in exactly 1 successful withdrawal and 99 "Insufficient Funds / Lock Timeout" rejections.

TR-05: Strict Cryptographic Idempotency Keys

- **Description (What):** Every financial request hitting the backend must include a unique, client-generated UUIDv4 Idempotency Key in the header. The system must cache this key for 24 hours. If a request arrives with a previously seen key, the system must bypass execution and instantly return the exact original cached response.
- **Rationale (Why):** If a buyer scans the QR code, but their phone loses 4G connection before receiving the "Success" message, they will tap "Scan" again. Idempotency guarantees the seller doesn't get paid twice from the buyer's wallet.
- **Acceptance Criteria:**
    - *Test 1:* Submitting the identical valid payload and Idempotency Key three times must execute exactly one database write, while returning HTTP 200 OK to all three requests.

TR-06: Distributed Saga Compensating Transactions

- **Description (What):** If the settlement process requires calls to multiple isolated microservices (e.g., Tax Service, Ledger Service, Notification Service), the system must implement a Saga pattern. If step 3 fails, the system must autonomously execute mathematically inverted "compensating transactions" to undo steps 1 and 2.
- **Rationale (Why):** Standard database rollbacks do not work across distributed microservices. If the Tax Service logs the deduction, but the Ledger Service crashes before releasing the funds, the system must actively reverse the tax log to maintain integrity.
- **Acceptance Criteria:**
    - *Test 1:* Simulating an intentional crash in the final payout microservice must automatically trigger the execution of reverse ledger entries, zeroing out the previously logged platform fees and taxes.

Phase 3: Hardware Failures & Safety Nets

TR-07: Deadlock Sweeper (Orphaned State Resolution)

- **Description (What):** The system must run an independent, asynchronous background daemon that constantly scans the ledger for locked financial rows or pending transactions that have exceeded a strict execution timeout (e.g., 5000 milliseconds).
- **Rationale (Why):** If a backend server's motherboard physically fries while it is halfway through processing a meetup QR scan, those funds will be permanently locked in a dead state. The sweeper detects the dead process, clears the lock, and gracefully reverts the state.
- **Acceptance Criteria:**
    - *Test 1:* Terminating the server process mid-transaction must result in the daemon identifying the orphaned lock and rolling the user's wallet back to its prior state within 60 seconds.

TR-08: External Gateway Circuit Breaker

- **Description (What):** The system must continuously monitor the error rate of external payment gateways (e.g., bank disbursement APIs). If the failure rate exceeds a predefined threshold (e.g., 5 failures in 10 seconds), the system must automatically "trip the circuit breaker" and queue all outgoing payouts internally until the external service recovers.
- **Rationale (Why):** Continuously hammering a downed banking API with live user transactions creates a massive backlog of failed financial states that require manual human reconciliation.
- **Acceptance Criteria:**
    - *Test 1:* Simulating an external bank API outage must trigger the circuit breaker, resulting in the user receiving an "Escrow Successful - Payout Queued" message rather than a hard crash.

Phase 4: Regulatory, AML & Auditing

TR-09: Cryptographic Ledger Hash Chaining

- **Description (What):** Every finalized financial ledger entry must store a cryptographic hash that is mathematically derived from its own transaction data AND the hash of the immediately preceding transaction in the database.
- **Rationale (Why):** This creates an immutable, blockchain-like audit trail. If a rogue database administrator attempts to manually alter a historical transaction to steal funds, it instantly breaks the hash chain for every subsequent transaction, triggering a massive internal alarm.
- **Acceptance Criteria:**
    - *Test 1:* Manually executing an UPDATE query directly on the database to change a user's balance must immediately cause the continuous audit script to flag a "Chain Integrity Failure."

TR-10: Independent Tax Segregation Wallets

- **Description (What):** Upon execution of the settlement phase, the transactional engine must physically route the calculated Withholding Tax (WHT) and ICT Sales Tax into isolated, mathematically distinct system wallets, rather than co-mingling them with the platform's operating revenue.
- **Rationale (Why):** Commingling user funds, platform revenue, and government tax liabilities is illegal in most financial jurisdictions and makes passing a regulatory audit impossible.
- **Acceptance Criteria:**
    - *Test 1:* Querying the platform's Operating_Revenue wallet must strictly return the sum of the Boli.pk platform fees, entirely exclusive of any WHT or ICT funds.

TR-11: Velocity-Based AML Freezes (Anti-Money Laundering)

- **Description (What):** The transactional engine must continuously calculate the 24-hour financial velocity (total value of items bought and sold) for every user. If a user exceeds a high-risk velocity threshold, the system must autonomously freeze their escrow withdrawals and flag the account for manual compliance review.
- **Rationale (Why):** To prevent bad actors from using the platform to rapidly launder stolen cash via fake listings before the system administrators notice.
- **Acceptance Criteria:**
    - *Test 1:* A user successfully settling Rs. 2,000,000 worth of escrow transactions within a 12-hour window must have their 11th payout automatically redirected to a holding account pending administrative clearance.

TR-12: Multi-Signature Administrative Overrides

- **Description (What):** Any administrative attempt to manually reverse a settled transaction, refund an escrow lock, or alter a user's financial balance must cryptographically require the explicit, logged approval of two separate high-level administrative accounts (Maker-Checker).
- **Rationale (Why):** A single customer support agent or rogue admin must never possess the unilateral power to move user money.
- **Acceptance Criteria:**
    - *Test 1:* Admin A triggering a manual refund must place the transaction in an AWAITING_AUTHORIZATION state. The funds must not move until Admin B supplies their cryptographic signature to approve the request.

TR-13: Continuous EOD Automated Reconciliation

- **Description (What):** The system must execute an automated End-of-Day (EOD) script that recalculates the entire platform's financial history from transaction #1 to the current moment, verifying that the computed balances match the actual database balances to the exact Paisa.
- **Rationale (Why):** This is the ultimate safety net. If a silent bug or race condition manages to bypass all real-time checks, the nightly reconciliation script catches the mathematical drift before the next business day begins.
- **Acceptance Criteria:**
    - *Test 1:* Modifying a user's balance by 1 Paisa without a corresponding ledger entry must trigger an immediate high-severity pager alert to the engineering team upon the execution of the EOD script.

TR-14: Immutable Payout Webhook Ingestion

- **Description (What):** When receiving external confirmation that funds have cleared a user's actual bank account, the system must process the payload, log the transition to CLEARED, and permanently lock the row.
- **Rationale (Why):** External banks can sometimes send delayed or conflicting webhooks hours later. The system must treat the initial valid confirmation as immutable truth to prevent double-crediting.
- **Acceptance Criteria:**
    - *Test 1:* Receiving a "Payment Pending" webhook from a bank 10 minutes *after* receiving a "Payment Cleared" webhook for the same transaction ID must be gracefully ignored and logged as an anomaly.

TR-15: Isolated Dispute Escrow Routing

- **Description (What):** If an active transaction is marked as DISPUTED, the locked funds must be automatically migrated from the standard Escrow_Liability ledger into an isolated Disputed_Liability ledger.
- **Rationale (Why):** Financial reporting. The company must be able to instantly report to investors or auditors exactly how much money is clearing normally versus how much is tied up in legal or user conflict.
- **Acceptance Criteria:**
    - *Test 1:* Changing a listing state to DISPUTED must simultaneously trigger an internal ledger transfer, zeroing out the amount in standard escrow and crediting the dispute ledger.

TR-16: Fractional Arbitration Disbursement

- **Description (What):** The transactional engine must support administrative split-settlements from the Disputed_Liability ledger, allowing authorized admins to disburse custom percentage splits of the locked funds back to both the buyer and the seller simultaneously.
- **Rationale (Why):** Not all disputes are binary (fraud vs. truth). If a laptop has a minor scratch not shown in the photos, but the buyer agrees to keep it for a 15% discount, the system must mathematically support dividing the single escrow lock into two distinct outbound transfers.
- **Acceptance Criteria:** An admin command to split a Rs. 100,000 disputed transaction "85/15" must strictly credit Rs. 85,000 to the seller's available balance, return Rs. 15,000 to the buyer's balance, and calculate zero platform fees for the reversed portion.

Security, Compliance & Infrastructure Requirements

SCR-01: Real-Time Collusion Detection (Shill Bidding Prevention)

- **Description (What):** The bidding engine must asynchronously route all incoming bid payloads through a continuous fraud-detection heuristic. The system must evaluate the bidder’s active IP address, hardware device fingerprint, and network BSSID against the corresponding data points of the listing's seller.
- **Rationale (Why):** To prevent market manipulation. A seller utilizing a secondary device on their home network to bid on their own auction artificially inflates the price, destroying buyer trust and violating platform integrity.
- **Acceptance Criteria:**
    - *Test 1:* A user attempting to submit a bid on a listing where the bidder's device fingerprint or active IP address matches the seller's active session data must have the bid silently dropped and their account flagged for administrative review.

SCR-02: Cryptographic Anonymization (Privacy Compliance)

- **Description (What):** Upon processing a legally mandated "Account Deletion" request, the data layer must permanently destroy all Personally Identifiable Information (PII), including names, phone numbers, NTNs, and biometric hashes. However, the system must substitute the user's primary identifiers with a persistent Deleted_Entity_UUID to preserve all associated financial ledgers and transaction histories.
- **Rationale (Why):** To resolve the paradox between data privacy laws (the Right to be Forgotten) and financial auditing. Hard-deleting a user's entire database footprint would irreversibly corrupt the platform's Zero-Sum double-entry ledger.
- **Acceptance Criteria:**
    - *Test 1:* Executing a compliant account deletion must result in a null or hashed state for all PII fields, while an execution of the End-of-Day (EOD) financial reconciliation script still mathematically balances to zero.

SCR-03: Layer-7 API Rate Limiting (Financial DDoS Protection)

- **Description (What):** The primary API Gateway must enforce strict, token-bucket rate limits categorized by both IP address and cryptographic Session ID. Endpoints that trigger third-party financial costs (e.g., OTP SMS dispatch, FBR NTN validation, AI Vision processing) must be isolated with highly restrictive cooldown thresholds.
- **Rationale (Why):** To prevent resource exhaustion and infrastructure bankruptcy. A malicious script hammering the SMS OTP endpoint 10,000 times a minute could incur massive third-party billing charges before a single valid transaction occurs.
- **Acceptance Criteria:**
    - *Test 1:* Transmitting 20 requests to the /api/v1/auth/send-otp endpoint from a single IP address within a 60-second window must trigger a strict HTTP 429 Too Many Requests response and temporarily blackhole the IP at the firewall level.

SCR-04: Disaster Recovery & High Availability (RPO/RTO)

- **Description (What):** The relational database architecture must maintain synchronous, cross-region replication. The infrastructure must legally guarantee a Recovery Point Objective (RPO) of exactly 0 seconds (zero accepted data loss) and a Recovery Time Objective (RTO) of strictly less than 60 seconds.
- **Rationale (Why):** If the primary physical data center experiences a catastrophic failure (e.g., fire, total grid loss) mid-auction, the system must autonomously failover to a geographic replica without losing a single Paisa or dropping a committed bid.
- **Acceptance Criteria:**
    - *Test 1:* Hard-terminating the primary database cluster during a simulated load test must result in the replica autonomously promoting itself to the primary write-node within 60 seconds, with the ledger reflecting every transaction committed prior to the exact millisecond of termination.

SCR-05: End-to-End Cryptographic Transit & Storage

- **Description (What):** All external and internal network communications must be strictly encrypted in transit using TLS 1.3 or higher. Furthermore, all sensitive user data (PII, financial ledgers, biometric hashes) must be encrypted at rest utilizing a dedicated Key Management Service (KMS) or Hardware Security Module (HSM) with automated key rotation every 90 days.
- **Rationale (Why):** If a malicious actor intercepts the network traffic or physically steals a hard drive from the data center, the data must remain mathematically useless to them.
- **Acceptance Criteria:**
    - *Test 1:* Any API request transmitted over HTTP or TLS 1.2 must be instantly dropped by the ingress controller before reaching the application layer.

SCR-06: Zero-Trust Internal Service Mesh

- **Description (What):** The backend infrastructure must implement a strict Zero-Trust architecture between all internal microservices. The AI Vetting microservice must not be allowed to communicate with the Financial Ledger microservice without explicitly authenticating via mutual TLS (mTLS) and presenting a valid, short-lived internal JWT token.
- **Rationale (Why):** If a hacker finds a vulnerability in the AI image-upload service, Zero-Trust ensures they are "boxed in." They cannot use the compromised AI server to start sending commands to the financial database because the database will reject the unauthenticated internal request.
- **Acceptance Criteria:**
    - *Test 1:* A simulated direct internal API call from the Notification Service to the Escrow Ledger Service without a valid mTLS certificate must return an HTTP 403 Forbidden.

SCR-07: Centralized SIEM Event Streaming

- **Description (What):** Every microservice, database, and API gateway must stream all authentication events, state changes, and error logs asynchronously to an isolated, centralized Security Information and Event Management (SIEM) cluster.
- **Rationale (Why):** When a breach or a massive bug occurs, engineers cannot waste time logging into 50 different servers to read text files. The SIEM provides a single, immutable pane of glass for security auditing and automated threat detection.
- **Acceptance Criteria:**
    - *Test 1:* A failed login attempt on the frontend must mathematically result in a corresponding alert populating within the isolated SIEM dashboard within 3000 milliseconds.

SCR-08: Ephemeral Secrets Management

- **Description (What):** The source code and database configurations must contain absolutely zero hardcoded passwords, API keys, or cryptographic seeds. All sensitive credentials must be dynamically injected into the application memory at runtime by an isolated, encrypted Secrets Manager.
- **Rationale (Why):** The most common way platforms get hacked is a junior developer accidentally uploading a file containing a live database password to a public GitHub repository. Ephemeral secrets make the source code inherently safe even if it leaks.
- **Acceptance Criteria:**
    - *Test 1:* A static code analysis scan of the entire codebase and environment configuration files must return zero instances of plaintext database URIs, Raast API keys, or FBR authentication tokens.

SCR-09: Automated CI/CD Vulnerability Blocking

- **Description (What):** The continuous integration/continuous deployment (CI/CD) pipeline must autonomously execute static application security testing (SAST) and software composition analysis (SCA) on every single code commit.
- **Rationale (Why):** Security cannot be an afterthought. If a developer attempts to merge code containing a known vulnerability (e.g., an outdated, exploitable open-source package), the deployment pipeline must act as an automated architect and reject the code.
- **Acceptance Criteria:**
    - *Test 1:* Attempting to merge a code branch that imports a dependency with a publicly known "Critical" CVE (Common Vulnerabilities and Exposures) must cause the automated build pipeline to fail and block the deployment to production.

SCR-10: Geographic Data Sovereignty

- **Description (What):** The infrastructure topology must explicitly restrict the physical storage and processing of all Pakistani citizen PII, tax records, and escrow ledgers to data centers physically located within the jurisdiction of Pakistan (or legally compliant regions as dictated by the FBR/State Bank).
- **Rationale (Why):** National compliance. Financial regulators strictly forbid the offshore storage of citizen tax and banking data. A failure here results in the platform losing its operating license immediately.
- **Acceptance Criteria:**
    - *Test 1:* An infrastructure audit must confirm that all primary and disaster-recovery database nodes containing NTN data are physically provisioned in approved, compliant regional zones.

Nonfunctional Requirements

Pillar 1: Performance & Scalability

NFR-01: The C10K Concurrency Ceiling

- **Description (What):** The primary WebSocket gateway must successfully maintain and actively route data for a minimum of 10,000 concurrent, full-duplex client connections on a single provisioned node without triggering an Out-Of-Memory (OOM) exception or dropping connections.
- **Rationale (Why):** Auctions are highly concentrated events. A popular listing will draw thousands of spectators into a single room in the final 60 seconds. The infrastructure must handle this traffic spike without buckling.
- **Acceptance Criteria:**
    - *Test 1:* Subjecting the bidding gateway to a synthetic load test of 10,000 simultaneous connections transmitting 1 payload per second must yield a 0% connection drop rate over a 5-minute sustained window.

NFR-02: Microsecond Read/Write Latency

- **Description (What):** The in-memory caching layer must process and resolve live auction state reads and transient bid writes with a P99 latency of strictly less than 15 milliseconds.
- **Rationale (Why):** To prevent client-side desynchronization. If the cache takes 200ms to register a bid, another user could place a mathematically invalid bid in the interim, corrupting the auction state.
- **Acceptance Criteria:**
    - *Test 1:* Executing an automated benchmarking script against the caching layer with 5,000 requests per second must return a 99th percentile response time of $\le$ 15ms.

NFR-03: AI Inference Throughput Constraint

- **Description (What):** The isolated computer-vision microservice must process, score, and return the JSON payload for a standard 5MB uploaded image in strictly less than 3,000 milliseconds under normal operating load.
- **Rationale (Why):** Sellers will abandon the listing process if they have to stare at a loading spinner for 30 seconds while the AI model evaluates their image.
- **Acceptance Criteria:**
    - *Test 1:* Submitting 50 simultaneous image payloads to the AI vetting endpoint must result in an average processing time of under 3000ms per image.

Pillar 2: Availability & Resilience

NFR-04: High Availability (Five Nines Target)

- **Description (What):** The core bidding engine and financial ledger routing services must be architected to achieve 99.99% uptime (allowing for a maximum of 4.38 minutes of downtime per month).
- **Rationale (Why):** If the server goes down during the final minute of a Rs. 500,000 auction, the highest bidder wins by default at a lower price, and the seller takes a massive financial hit due to system failure.
- **Acceptance Criteria:**
    - *Test 1:* The system architecture must include redundant, load-balanced instances of the core gateway, ensuring that the hard termination of Node A results in zero dropped bids as traffic seamlessly shifts to Node B.

NFR-05: Mean Time To Recovery (MTTR)

- **Description (What):** In the event of a catastrophic container or service crash, the infrastructure orchestration layer must detect the failure, spin up a replacement container, and restore service routing within a maximum MTTR of 30 seconds.
- **Rationale (Why):** Failures are inevitable. The system's ability to self-heal faster than a user can refresh their browser is critical for perceived reliability.
- **Acceptance Criteria:**
    - *Test 1:* Manually executing a kill -9 command on the primary backend process must trigger an automated container respawn and health-check clearance within 30 seconds.

Pillar 3: Regulatory & Data Lifecycle

NFR-06: Statutory Data Retention (SBP/FBR Compliance)

- **Description (What):** The data architecture must enforce a strict, automated immutability policy retaining all transactional ledgers, tax deductions, and KYC identification records for a minimum of 5 years.
- **Rationale (Why):** Direct legal compliance. The State Bank of Pakistan's AML/CFT framework mandates a 5-year retention period for all financial tracking to assist in forensic auditing, even if the user explicitly deletes their frontend account.
- **Acceptance Criteria:**
    - *Test 1:* An automated database chron-job attempting to purge transaction records older than 4 years and 364 days must be hard-rejected by the database policy constraints.

NFR-07: Cold Storage Archival Strategy

- **Description (What):** Log data and finalized escrow receipts older than 90 days must be autonomously migrated from high-cost hot-storage databases to encrypted, low-cost cold storage (e.g., AWS S3 Glacier or equivalent).
- **Rationale (Why):** To maintain the microsecond read/write latency (NFR-02) of the active database. Keeping 5 years of historical records in active memory will bloat the indexes and destroy system speed.
- **Acceptance Criteria:**
    - *Test 1:* Verifying the primary active database size must show zero transaction records older than 90 days, while querying the archival data lake must successfully retrieve the older records within 5 seconds.

Pillar 4: Observability & Telemetry

NFR-08: Distributed Request Tracing

- **Description (What):** The infrastructure must append a unique cryptographic Trace-ID to every incoming API request and propagate that exact ID across all internal microservice boundaries (Gateway $\rightarrow$ AI Service $\rightarrow$ Ledger).
- **Rationale (Why):** If a transaction fails, engineers cannot waste hours guessing which microservice dropped the ball. Tracing provides an exact X-ray of the data's path through the system.
- **Acceptance Criteria:**
    - *Test 1:* Searching the central logging dashboard for a specific Trace-ID must return a chronological list of execution steps spanning at least three distinct internal microservices.

NFR-09: Centralized Audit Logging Latency

- **Description (What):** Every microservice must asynchronously stream its state changes and error logs to a centralized Security Information and Event Management (SIEM) cluster with an ingestion latency of less than 2,000 milliseconds.
- **Rationale (Why):** In the event of an active cyber-attack or fraud attempt, security administrators must have real-time visibility into the system state, not logs that are 10 minutes delayed.
- **Acceptance Criteria:**
    - *Test 1:* Triggering an intentional "Invalid Password" error on the frontend must result in the corresponding alert populating in the centralized SIEM dashboard in under 2 seconds.

Pillar 5: Maintainability & Code Quality

NFR-10: Strict Test Coverage Thresholds

- **Description (What):** The Continuous Integration (CI) pipeline must enforce a hard quality gate requiring a minimum of 80% overall line coverage, and strictly 100% test coverage on all mathematical operations within the Financial Ledger module.
- **Rationale (Why):** To prevent tech debt and catastrophic bugs. Developers must not be physically allowed to merge code that touches user money unless a suite of automated tests proves the math works.
- **Acceptance Criteria:**
    - *Test 1:* Attempting to merge a pull request containing untested code in the EscrowSettlement module must cause the CI pipeline to fail and block the deployment.

NFR-11: Self-Contained Simulation Execution

- **Description (What):** The entire platform architecture must be capable of executing in a completely isolated, self-contained local environment without requiring external internet access or live third-party financial API connections.
- **Rationale (Why):** To guarantee a flawless, zero-latency demonstration during your academic viva. Relying on live university Wi-Fi during a high-stakes demo introduces uncontrollable failure points.
- **Acceptance Criteria:**
    - *Test 1:* Disconnecting the host machine from the internet and executing a single deployment script (e.g., docker-compose up) must boot the entire platform, allowing a full simulated bid-to-escrow-settlement workflow.

NFR-12: Agile Delivery Timebox

- **Description (What):** The core components—specifically the double-entry ledger, the concurrent bidding engine, and the QR settlement handshake—must be architected to fit within a strict 14-day development sprint limit.
- **Rationale (Why):** The architecture must prioritize the deployment of the critical path over secondary features to meet the university Complex Engineering Problem (CEP) constraints.
- **Acceptance Criteria:**
    - *Test 1:* The project management timeline must demonstrate a complete, functional path from code initialization to the localized deployment within 14 calendar days.

Pillar 6: Interoperability & Frontend Usability

NFR-13: OpenAPI 3.0 Contract Standardization

- **Description (What):** All external-facing REST endpoints must be strictly documented and validated against the OpenAPI 3.0 specification to ensure seamless integration with Raast P2M APIs and external bank webhooks.
- **Rationale (Why):** Financial institutions will not integrate with a platform that has poorly documented or non-standard API endpoints.
- **Acceptance Criteria:**
    - *Test 1:* Running an automated OpenAPI linter against the backend routing code must return zero critical validation errors.

NFR-14: Stateless API Versioning

- **Description (What):** The API gateway must enforce URI-based versioning (e.g., /api/v1/bid) and guarantee backward compatibility for at least one deprecated major version.
- **Rationale (Why):** If you update the backend database logic, you cannot instantly break the mobile apps installed on users' phones who haven't downloaded the latest app store update.
- **Acceptance Criteria:**
    - *Test 1:* Deploying version v2 of the settlement API must allow legacy mobile clients transmitting payloads to the v1 endpoint to successfully complete their transactions without crashing.

NFR-15: Optimistic Animation Thresholds

- **Description (What):** The client interface must trigger tactile UI feedback (e.g., button depression states, haptics) within 16 milliseconds (1 visual frame at 60fps) of a user interaction, decoupled from the network request lifecycle.
- **Rationale (Why):** To make the application feel physically responsive and native, completely masking the unavoidable 50ms round-trip network lag during bid submission.
- **Acceptance Criteria:**
    - *Test 1:* A high-speed UI profiling tool must verify that the CSS active state or frame animation begins within 16ms of the DOM onClick event firing.

Task 2:

- Boli.pk
  - SCRUM-1 Functional Requirements
    - SCRUM-3 TR-01: Integer-Based Currency Storage
    - SCRUM-4 TR-02: Zero-Sum Double-Entry Constraint
    - SCRUM-5 DR-11: Two-Phase Commit (2PC) Settlement
    - SCRUM-6 DR-12: Cryptographic Non-Repudiation
    - SCRUM-7 SCR-01: Real-Time Collusion Detection
    - SCRUM-8 PR-08: Network-Adaptive Payload Delivery
    - SCRUM-9 IR-07: AI Service Timeout Constraint
  - SCRUM-2 Non-Functional Requirements
    - SCRUM-10 NFR-01: The C10K Concurrency Ceiling
    - SCRUM-11 NFR-02: Microsecond Read/Write Latency
    - SCRUM-12 NFR-04: High Availability (Five Nines Target)
    - SCRUM-13 NFR-06: Statutory Data Retention
    - SCRUM-14 NFR-10: Strict Test Coverage Thresholds
    - SCRUM-15 NFR-09: Centralized Audit Logging Latency
    - SCRUM-16 NFR-15: Optimistic Animation Thresholds

Non-Functional Requirements

To Do | Edit

Link Type	Key	Summary	Status	Ori...	Action
	SCRUM-10	NFR-01: The C10K Concurrency Ceiling	TO DO	TREE	•••
	SCRUM-11	NFR-02: Microsecond Read/Write Latency	TO DO	TREE	•••
	SCRUM-12	NFR-04: High Availability (Five Nines Target)	TO DO	TREE	•••
	SCRUM-13	NFR-06: Statutory Data Retention	TO DO	TREE	•••
	SCRUM-15	NFR-09: Centralized Audit Logging Latency	TO DO	TREE	•••
	SCRUM-14	NFR-10: Strict Test Coverage Thresholds	TO DO	TREE	•••
	SCRUM-16	NFR-15: Optimistic Animation Thresholds	TO DO	TREE	•••

Functional Requirements

To Do | Edit

☑	SCRUM-3	TR-01: Integer-Based Currency Storage	=	TO DO	TREE	•••
☑	SCRUM-4	TR-02: Zero-Sum Double-Entry Constraint	=	TO DO	TREE	•••
☑	SCRUM-5	DR-11: Two-Phase Commit (2PC) Settlement	=	TO DO	TREE	•••
☑	SCRUM-6	DR-12: Cryptographic Non-Repudiation	=	TO DO	TREE	•••
☑	SCRUM-7	SCR-01: Real-Time Collusion Detection	=	TO DO	TREE	•••
☑	SCRUM-8	PR-08: Network-Adaptive Payload Delivery	=	TO DO	TREE	•••
☑	SCRUM-9	IR-07: AI Service Timeout Constraint	=	TO DO	TREE	•••